TinyIDS: the project explained

What the proposal asks for, what every term means, and how we will do it

TinyIDS = a tiny **IDS (Intrusion Detection System)**: a program that watches network traffic and raises an alert when it sees an attack. It must be small and fast enough to run on a **Raspberry Pi 4** acting as the **IoT (Internet of Things)** gateway.

Contents

1 The two halves of the project: offline and live **2**

2 Machine learning in plain English **3**

3 Making models small: compression **5**

4 SHAP: explaining every alert **5**

5 The three research questions (RQ) in plain words **6**

6 Targets from the proposal (Evaluation metrics) **6**

7 The testbed: build everything on the laptop first **7**

8 Performing the attacks **9**

9 Planned: laptop as "master" of the NodeMCUs (simulated botnet) **10**

10 Next steps **10**

Appendix: full forms of all abbreviations **12**

1 The two halves of the project: offline and live

The proposal asks for two different kinds of work. Mixing them up causes most of the confusion.

(a) Offline: on the dataset

> *"Benchmark at least four classifiers on CICIoT2023 for binary (benign vs attack), 8-class and 34-class detection."*

(Objective 1)

CICIoT2023 = Canadian Institute for Cybersecurity IoT dataset 2023. Its `test.csv` already contains **all 8 families**: DDoS, DoS, Mirai, Recon, Spoofing, Web-based, BruteForce and Benign. You do *not* perform these attacks yourself: the dataset authors already ran them in their lab and recorded the traffic. Your model is trained on `train.csv` and scored on `test.csv`. This gives the "accuracy" numbers of the report.

(b) Live: on your testbed

> *"An attacker laptop generates SYN/UDP floods (hping3), port scans (nmap) and MQTT brute force."*

(Methodology, step 5)

This is the **only** live attack list in the proposal. It covers 4 of the 8 families:

Live attack	Tool	Dataset family
SYN flood	hping3	DDoS / DoS
UDP flood	hping3	DDoS / DoS
Port scan	nmap	Recon
MQTT brute force	small Python script	BruteForce
Normal sensor traffic	NodeMCUs	Benign

Mirai, Spoofing and Web-based are evaluated only on the dataset. Nothing is missing. (Optional extra later: ARP spoofing can be done live on your own hotspot.)

What the acronyms mean

- **DoS** = Denial of Service; **DDoS** = Distributed Denial of Service (many attackers at once).
- **SYN** = synchronize (the "let's connect" TCP packet); **TCP** = Transmission Control Protocol; **UDP** = User Datagram Protocol.
- **hping3** = a packet generator ("ping on steroids"); **nmap** = Network Mapper, a port scanner.
- **MQTT** = Message Queuing Telemetry Transport; **CoAP** = Constrained Application Protocol.

2 Machine learning in plain English

The basics

- **Feature** = one input column (e.g. `Rate`, `syn_flag_number`). CICIoT2023 has 46.
- **Label** = the correct answer for a row (e.g. `DDoS-SYN_Flood`).
- **Classifier** = a program that looks at a row's features and predicts its label. It *learns* this by looking at millions of labelled rows ("training").
- **Binary / 8-class / 34-class** = how detailed the answer must be: attack-or-benign, which family, or which exact attack.

The four models the proposal asks for

"Train Random Forest, XGBoost/LightGBM, a 3-layer MLP and a small 1D-CNN."

(Methodology, step 2)

Model	What it is
Decision tree (building block)	A flowchart of yes/no questions learned from data: "`Rate` $>$ 5000? $\rightarrow$ `syn_flag_number` $>$ 0.8? $\rightarrow$ SYN flood". Easy to understand, but one tree alone makes mistakes.
Random Forest (RF)	**Hundreds of decision trees that vote.** Each tree is trained on a slightly different random part of the data; the majority answer wins. Accurate, robust, easy to use.
XGBoost = eXtreme Gradient Boosting	Also many trees, but built **one after another**: each new tree focuses on fixing the mistakes of the previous ones ("boosting"). Usually the most accurate on table data like ours.
LightGBM = Light Gradient Boosting Machine	Microsoft's faster, lighter version of the same boosting idea. The proposal says "XGBoost/LightGBM": **pick one**. We will use LightGBM (fast, small models).
MLP = Multi-Layer Perceptron	The classic **ANN (Artificial Neural Network)**: layers of "neurons" connected by weights. "3-layer" = three layers.
1D-CNN = one-dimensional Convolutional Neural Network	A neural network that slides a small window **along a row of numbers** (1-D) instead of across an image (2-D). For table data it is usually *not* better than trees; it is included because many IDS papers use it, so it gives a fair comparison point.

Expected result: the tree models (RF, LightGBM) will probably be the most accurate *and* the fastest/smallest. That is a perfectly good finding.

How models are scored

- **Accuracy** = fraction of rows predicted correctly. Misleading on unbalanced data (predicting "DDoS" for everything already gives 73%).
- **Precision** = of the rows the model called "attack X", how many really were X.
- **Recall** = of all real "attack X" rows, how many the model found.
- **F1-score** = one number combining precision and recall (their harmonic mean), from 0 to 1.

- **Macro-F1** = the F1 of every class **averaged equally**, so rare attacks (Web, BruteForce) count as much as DDoS. The proposal uses this.
- **Confusion matrix** = a table of "real class vs predicted class" showing exactly which attacks get mixed up.
- **FPR (False Positive Rate)** = how often normal traffic is wrongly flagged as an attack.

3 Making models small: compression

The Pi is much weaker than a laptop, so the trained models are shrunk before deployment.

Term	Meaning
Pruning	Cutting away the parts of a model that matter least. Trees: fewer trees and less depth. MLP: remove the weakest connections between neurons.
Quantization	Storing the model's numbers with fewer bits, e.g. 32-bit decimal numbers → **INT8** (8-bit integers). About 4× smaller and faster, with a very small accuracy loss.
Quantized tree ensemble	"Ensemble" = many trees together (RF or LightGBM). Quantized = its thresholds/values stored in fewer bits, and usually pruned too.
TFLite = TensorFlow Lite	Google's format for running neural networks (MLP, CNN) on small devices like phones and the Pi.
ONNX = Open Neural Network Exchange	An open, universal model file format. **ONNX Runtime** runs ONNX models fast on almost any device, including the Pi. Tree models can also be exported to ONNX.

4 SHAP: explaining every alert

SHAP = **SHapley Additive exPlanations**. It is based on Shapley values from game theory: it treats the features like players in a team and fairly splits "credit" for the decision among them.

For **one** prediction it gives every feature a score: how much it pushed the decision towards (+) or away from (−) the predicted class. Example alert:

```
ALERT: DDoS-SYN_Flood
  because  syn_flag_number = 1.0      (+0.45)
           Rate = 98,000 packets/s    (+0.31)
           ack_flag_number = 0.0      (+0.12)
```

These are the **"top 3 features"** the dashboard must show. **TreeSHAP** is the fast, exact version of SHAP for tree models (RF, LightGBM, XGBoost).

SHAP has **two uses** in this project:

1. **Explain live alerts** (RQ3, Objective 5).
2. **Rank features** (RQ1, Objective 2): average the absolute SHAP scores over many rows → a ranking of the most important features → retrain using only the top 5 / 10 / 15 / 20.

5 The three research questions (RQ) in plain words

RQ1: how many features do we really need?

> *"How few flow features (out of 46 in CICIoT2023) are needed to keep macro-F1 within 2% of the full-feature model?"*

1. Train with **all 46 features**. Say macro-F1 = 0.90.
2. Retrain with only the top 5, 10, 15, 20 features (ranked by SHAP).
3. Find the **smallest** number that still reaches **at least 0.88** (within 2 points of 0.90).

It does **not** mean "98% accuracy". Fewer features = faster feature extraction on the Pi. Result: one plot, macro-F1 vs number of features.

RQ2: which compressed model is best on the Pi?

> *"Which compressed model (quantized tree ensemble, pruned MLP, small 1D-CNN) gives the best accuracy per millisecond and per megabyte on the Pi?"*

Compare the three compressed models on the Pi: accuracy vs speed (milliseconds per prediction) vs size (megabytes). Result: one benchmark table.

RQ3: can SHAP keep up?

> *"Can SHAP-based explanations be produced on the gateway fast enough to accompany live alerts?"*

Measure how long TreeSHAP takes per alert on the Pi. Target: under 50 ms. Result: one number (plus a few examples).

6 Targets from the proposal (Evaluation metrics)

Category	What is measured	Target
Detection	macro-F1, per-class recall, FPR	**Binary F1** $\geq$ **0.98**; 8-class within 2% of full-feature model
Efficiency	model size, peak RAM on the Pi	$<$ **1 MB** model; $<$ **100 MB** RAM
Speed	latency per flow, throughput	$<$ **5 ms** per flow; $\geq$ **500 flows/s**
Energy	mJ (millijoules) per inference	just report it for every model
Explainability	extra SHAP time per alert	$<$ **50 ms**

Note: RAM is in **megabytes**, not kilobytes. **RAM** = Random Access Memory. For a Pi 4 with 4 GB RAM these targets are relaxed; tree models easily fit in under 1 MB.

7 The testbed: build everything on the laptop first

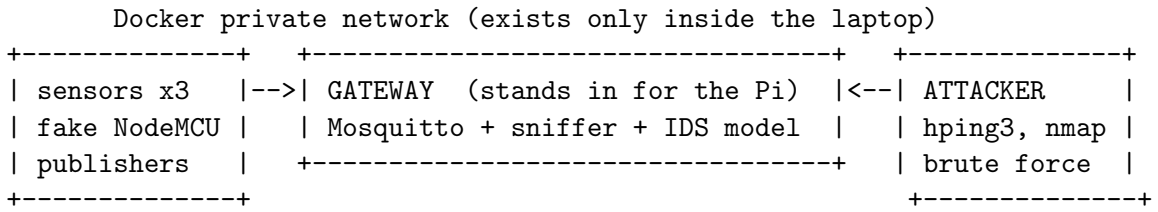
The plan

Phase	What	Where	Needs lab/Pi?
1. Train	Train RF, LightGBM, MLP, 1D-CNN on CICIoT2023 (binary + 8-class), SHAP ranking, retrain on top-k features, save model files.	Laptop / Colab	No
2. Laptop testbed	Fake gateway + fake sensors + attacker as Docker containers; run attacks; check detection.	Laptop (Docker)	No
3. Pi	Copy the same code to the Pi, real NodeMCUs, measure size / RAM / speed / energy.	Lab	**Yes**
4. Finish	Dashboard with top-3 SHAP features, demo video, report.	Laptop	No

Only phase 3 needs the lab. The Pi runs Linux and Python like the Docker containers, so the code moves over almost unchanged. (**Colab** = Google Colaboratory, free online Python notebooks with a GPU, Graphics Processing Unit.)

The laptop testbed (phase 2)

Docker runs small isolated Linux "machines" (**containers**) on the laptop, joined by a **private virtual network**:



- **Gateway**: runs Mosquitto (the MQTT broker), captures every packet, builds the features (Rate, SYN fraction, ...) for each window of packets, and asks the model "attack or not?". Alerts go to the dashboard with the top-3 SHAP features.
- **Sensors**: small Python scripts publishing fake temperatures over MQTT, like the NodeMCU test.
- **Attacker**: has hping3, nmap and an MQTT brute-force script.

Big advantage: attack traffic never leaves the laptop, so it is safe to run anywhere (hostel, home) without touching the college network. Already available on the laptop: Docker Desktop, **WSL** (Windows Subsystem for Linux) with Ubuntu, Wireshark/Npcap, 15 GB RAM.

What can be done without the lab or the Pi

- All of phase 1 (training, SHAP, feature selection, compression, accuracy tables and plots).
- All of phase 2 (live detection on the Docker testbed, all four live attacks).
- The dashboard and most of the report.

- The NodeMCUs can join the laptop testbed over a phone hotspot (laptop runs the broker and IDS) whenever you have them with you.

Only the **Pi-specific measurements** (model speed, RAM and energy on the Pi) need the Pi.

8 Performing the attacks

No device has to be "compromised"

The attacker simply sends bad traffic **at** the gateway, the way a real attacker would. The gateway does not have to be insecure: a flood still hits it, and the IDS's job is to **notice**.

Attack	Tool / command idea	What happens
SYN flood	`hping3 -S --flood -p 1883 <gateway>`	Thousands of "let's connect" (SYN) packets per second that never complete the handshake.
UDP flood	`hping3 --udp --flood <gateway>`	A blast of junk UDP packets.
Port scan	`nmap <gateway>`	Knocks on every port to see which are open (closed ports reply with RST).
MQTT brute force	Python script	Tries many username/password pairs on Mosquitto (needs a password set on the broker).

What the professor probably meant by "compromise a device"

Real botnets like Mirai first take over IoT devices and then use them to flood a target. You do not have to reproduce that: the dataset already contains Mirai traffic. But you *can* show the idea safely with your own NodeMCUs (next section).

Can we install real Mirai on the NodeMCU?

No. Mirai is Linux malware for cameras and routers that run a small Linux system. The **ESP8266** chip on the NodeMCU does not run Linux, so Mirai cannot run on it. Real Mirai would also automatically try to break into other devices on the network, which on the college network would be a genuine attack on other people's devices. We imitate its behaviour instead.

Rules

- Attack **only your own setup**: the Docker network on the laptop, or the Pi + NodeMCUs on your own phone hotspot.
- **Never** run floods or scans on the college network: IT will see a real attack.

9 Planned: laptop as "master" of the NodeMCUs (simulated botnet)

The idea

This imitates how Mirai controls infected devices, using only your own hardware:

1. Each NodeMCU runs a sketch that behaves like a **normal sensor**: it publishes data every few seconds.
2. It also **listens** on an MQTT command topic, e.g. `cmd/all`.
3. From the laptop (the "master", like Mirai's **C2** = Command-and-Control server) you publish a command, e.g.
   {`"attack":"udp_flood","seconds":10`}
4. Every NodeMCU suddenly starts attacking the gateway at the same time ⇒ a small **DDoS** from "compromised sensors".
5. After the time is up (or on a `stop` command) they go back to being normal sensors.
6. The IDS should flag the flood and show which features gave it away.

Yes, this is possible and it makes a strong demo: normal traffic → command → coordinated attack → alert.

What a NodeMCU can and cannot do

Attack from a NodeMCU	**Possible?**	**Note**
UDP flood	Yes	Like Mirai's `udpplain`. A few thousand packets/s per board.
TCP connection flood	Yes	Opens many real connections quickly.
Raw SYN flood with fake IPs	No	The ESP8266 cannot build raw packets; use hping3 on the laptop for this.
HTTP (web request) flood	Yes, if the gateway runs a web page	Optional.

Built-in safety (part of the plan)

- The attack target is **fixed in the sketch** (only the gateway's IP), not taken from the command.
- Every attack has a **maximum duration** (e.g. 30 s) and stops by itself.
- Use it only on your own hotspot / testbed network.

Where it fits

After phase 2 works with hping3, we add this as the "compromised IoT device" demo in phase 3 (or on the laptop testbed with NodeMCUs on a hotspot).

10 Next steps

1. **Training script**: load a sample of `train.csv`, keep features we can compute live, train RF + LightGBM + MLP + 1D-CNN for binary and 8-class, report macro-F1 and confusion matrices

on `test.csv`, save the models.

2. SHAP feature ranking and retraining with top 5/10/15/20 (RQ1 plot).
3. Docker testbed with gateway, sensors and attacker.
4. Live detection + SHAP alerts + dashboard.
5. Pi deployment and benchmark table (RQ2, RQ3).
6. Optional: master-controlled NodeMCU botnet demo.

Appendix: full forms of all abbreviations

Abbreviation	Full form / meaning
ACK	Acknowledge (TCP flag: "got it")
ANN	Artificial Neural Network
ARP	Address Resolution Protocol ("who has this IP address?")
C2	Command-and-Control (the server that gives orders to a botnet)
CICIoT2023	Canadian Institute for Cybersecurity IoT dataset 2023
CNN / 1D-CNN	Convolutional Neural Network / one-dimensional CNN
CoAP	Constrained Application Protocol
DDoS	Distributed Denial of Service
DNS	Domain Name System (turns names like google.com into IP addresses)
DoS	Denial of Service
ESP8266	The Wi-Fi microcontroller chip on the NodeMCU (made by Espressif)
F1	F1-score (harmonic mean of precision and recall)
FPR	False Positive Rate
GPU	Graphics Processing Unit
GRE	Generic Routing Encapsulation (a tunnelling protocol, used by Mirai floods)
IAT	Inter-Arrival Time (gap between two packets)
ICMP	Internet Control Message Protocol (ping)
IDS	Intrusion Detection System
INT8	8-bit integer
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation (text format for data)
LightGBM	Light Gradient Boosting Machine
MITM	Man-in-the-Middle
MLP	Multi-Layer Perceptron
MQTT	Message Queuing Telemetry Transport
NodeMCU	Node MicroController Unit (the ESP8266 development board)
ONNX	Open Neural Network Exchange
RAM	Random Access Memory
RF	Random Forest
RQ	Research Question
RSSI	Received Signal Strength Indicator
RST	Reset (TCP flag: "abort this connection")
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language (databases)
SYN	Synchronize (TCP flag: "let's connect")
TCP	Transmission Control Protocol

Abbreviation	Full form / meaning
TFLite	TensorFlow Lite
TTL	Time To Live
UDP	User Datagram Protocol
WSL	Windows Subsystem for Linux
XGBoost	eXtreme Gradient Boosting
XSS	Cross-Site Scripting